MAS1004 Data & AI, 2026 Fall

Assignment 2: Long report

Name: 이다윗 Student ID: 20231800

리더보드 닉네임: `rokafsogang` 레포: `https://github.com/rokafsogang/mas1004-assignment2`
마지막 릴리즈: Track A `track-a-3` (balanced + 사진 115장 제외), Track B `track-b-2` (MobileNetV3-Large, 384px).

학습 환경. Track A는 MacBook Air(M5)의 GPU(`mps`)에서, Track B는 Colab GPU(`cuda`)에서 학습했다. torch 2.6은 `mps`에서 `official.py`가 요청하는 `channels_last` 메모리 형식 때문에 첫 backward에서 "view size is not compatible with input tensor's size and stride" 오류로 멈췄다. `official.py`는 고치지 않고, `.to(memory_format=torch.channels_last)`만 아무 일도 하지 않게 만든 래퍼 `scripts/official_mps.py`로 실행했다. recipe, 사진 선택, 예산(249,984장), seed, 검증 방식은 `official.py` 그대로이고, 텐서의 메모리 배치만 다르다. 채점기는 `cuda`에서 float16으로 학습하므로 내 검증 점수와 채점기 점수는 조금 다를 수 있다.

Track A

tag	이전 실행에서 바뀐 것	validation
baseline (seed 0)	starter recipe 그대로, 학습 사진 44,257장 전부	0.4961
baseline (seed 1)	seed만 0 → 1 (우연에 의한 흔들림 측정)	0.4956
balanced	`sample_weights`: 각 사진에 1/(그 도시 사진 수), 도시를 같은 빈도로 뽑음	0.5076
balanced_drop	사진을 보고 고른 115장 제외 (44,257 → 44,142장), recipe는 balanced 그대로	0.5099
no_flip	`T.RandomHorizontalFlip()` 제거, 사진 선택은 balanced_drop 그대로	0.5001

도시별 정답률	seoul	busan	tokyo	osaka	taipei	평균
baseline	0.595	0.386	0.624	0.344	0.531	0.496
balanced	0.496	0.471	0.520	0.435	0.616	0.508
balanced_drop	0.536	0.459	0.517	0.423	0.614	0.510
no_flip	0.550	0.430	0.511	0.419	0.590	0.500
학습 사진 수	11,274	5,191	13,347	7,200	7,245	

seed에 따른 흔들림. 같은 starter recipe를 seed 0과 1로 학습하니 0.4961과 0.4956으로 평균은 0.0005 차이였다. 그러나 도시별로는 오사카 0.344 → 0.367(+0.024), 도쿄 0.624 → 0.607(−0.017)처럼 더 크게 움직였다. 결과를 어떻게 보는지(내 말): "같은 설정인데도 값이 살짝 다른걸 보면 기계라고 그 전껄 100% 같게 판독하지 못한다는 걸 알았다. 사람처럼 얘네도 시행마다 살짝씩 다르단걸". README에 따르면 같은 recipe도 다시 학습하면 2–3점 움직일 수 있다. 아래의 각 변경은 seed 0으로 한 번씩만 학습했으므로, 이 흔들림보다 작은 차이는 우연일 수 있다.

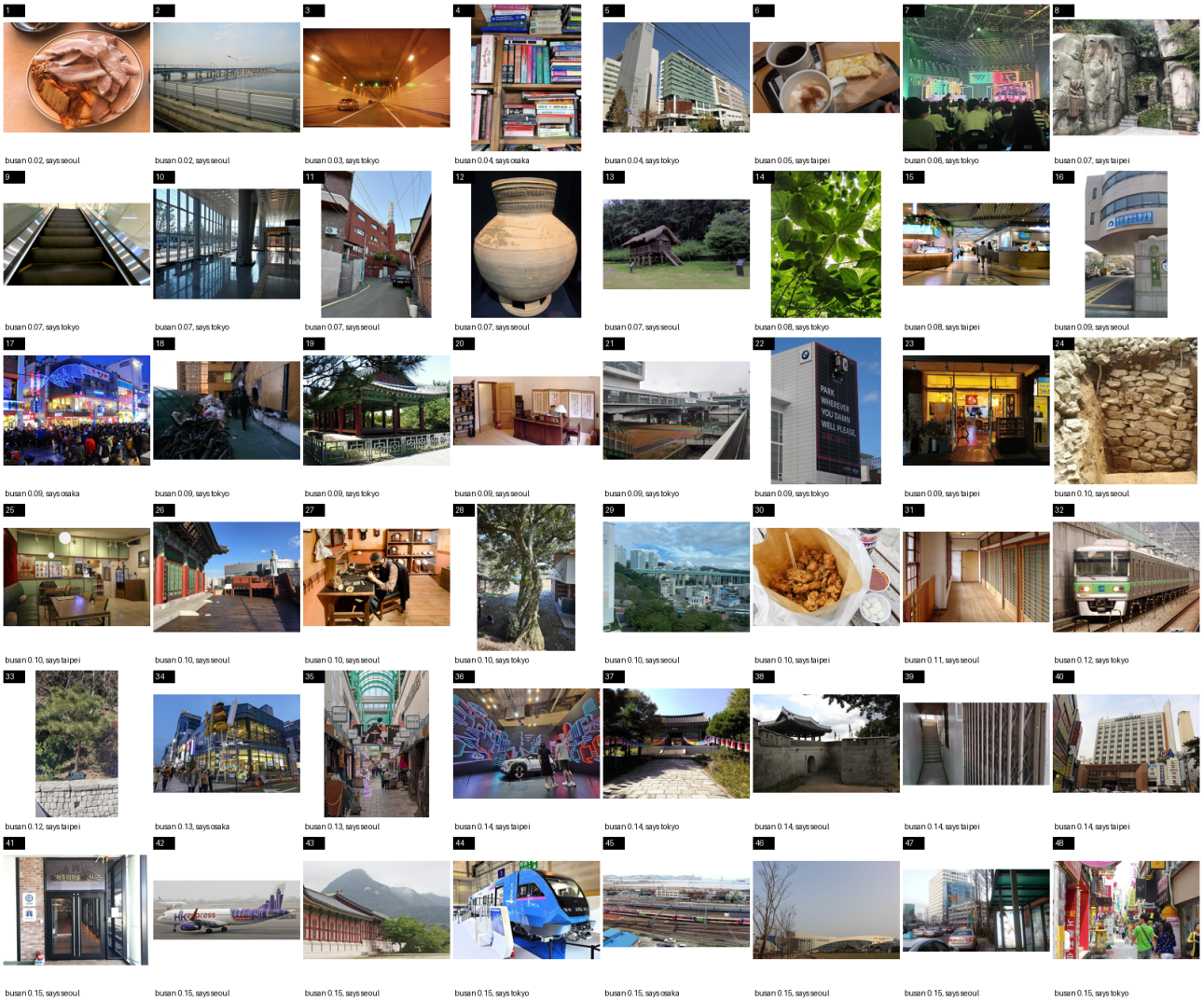
balanced. 학습 사진 수가 도쿄 13,347장, 부산 5,191장으로 2.6배 차이가 난다. 왜 골랐는지(내 말): "어차피 최대 양의 사진 (도쿄)꺼를 해야하는데 그거만 13,347할게 아니라 그 시간에 부산도 여러번 보면 정확도가 올라갈거라 생각해서 밸런스 선택". 평균은 0.496에서 0.508로 올랐다. 부산(+0.085), 오사카(+0.091), 타이베이(+0.085)는 올랐고 서울(−0.099), 도쿄(−0.104)는 내려갔다. 검증 사진 12,077장에 대한 모델의 답을 세어 보면, baseline은 "부산"이라고 938번 답했고(실제 부산 사진 1,390장) "도쿄"라고 4,320번 답했다(실제 3,881장). balanced는 "부산" 1,362번, "도쿄" 3,318번으로, 사진이 많은 도시 쪽으로 쏠리던 답이 고르게 바뀌었다 (`results/*/result.json`의 confusion matrix). 결과를 어떻게 보는지(내 말): "서울 도쿄를 제외한 나머지 도시는 베이스라인보다 높은 정확도를 보여줬는데 그 이유는 아무래도 사진 양이 적었기에 여러번 학습하여 정확도가 올라갔다고 보여짐".

no_flip. 좌우반전을 하면 간판 글자가 거울 글씨가 되고 차가 다니는 쪽이 바뀐다. 왜 골랐는지(내 말): "좌우반전을 하면 너 말대로 일본과 같은 반대인 나라에 대한 오류가 생길거라 생각이 들어". 다른 후보(더 강한 확대 자르기, label smoothing 끄기)를 고르지 않은 이유(내 말): "확대해봤자 픽셀이 더 세분화되는거 아니니 도움 안될거같고 100%라는건 세상에 없어". 결과: 0.5099 → 0.5001(−0.0098)로 내려갔다.

좌측통행인 도쿄(−0.007)와 오사카(−0.004)도 오르지 않았고, 부산(−0.029)과 타이베이(−0.024)가 가장 많이 내려갔으며 서울만 +0.014 올랐다. 결과를 어떻게 보는지(내 말): “예상과 달리 떨어졌고, 이유는 모르겠다”. 그래서 릴리즈하지 않았고 `track_a/recipe.py`는 track-a-3의 버전으로 되돌렸다.

최종 recipe와 리더보드. 마지막 Track A 릴리즈는 `track-a-3`(balanced + 115장 제외, validation 0.5099)이다. 리더보드 점수는 이 보고서를 쓴 시점(11월 9일 밤)에 아직 공개되지 않았다. 매일 자정 채점 결과가 다음 날 아침에 올라오고, 나는 11월 9일에 처음 릴리즈했기 때문이다. 그래서 검증 점수와 리더보드 점수를 나란히 비교하지 못했다.

사진을 보고 내린 결정



무엇을 봤나. baseline 모델로 `curate.py suspects results/baseline_seed0/model.onnx --every 4`를 돌려, 각 도시 학습 사진 중 모델이 그 도시일 확률을 가장 낮게 준 48장씩(위 그림은 부산)을 봤다. 처음 도시별 사진(`curate.py look`)을 봤을 때 도시를 구분하는 단서로 적은 것(내 말): “서울 잠실 주경기장을 통해 알수 있음, 부산 - 해운대 모래사장을 통해 알 수 있음, 도쿄 - JR전철을 통해(초록색), 오사카 - 도톤보리 수로를 통해, 타이페이 - 한자와 타이페이101을 통해 알 수 있음”.

규칙 (빼기 전에 정함, 내 말). “사람사진과 음식 사진 그 두개는 어디 나라인지정도까진 추측이 가능해도 도시를 알기는 어려움 그리고 의미없는 자동차 이런거 다 없앰”.

무엇을 바꿨나. 이 규칙에 맞는 사진을 시트 번호로 직접 골라 115장을 뺐다(서울 25, 부산 29, 도쿄 26, 오사카 25, 타이베이 10). `curate.py count`의 출력:

```
city      selected     of   by source
seoul        11249  11274   commons 6746, osv5m 319, youtube 4184
```

```
busan         5162   5191   commons 3459, youtube 1703
tokyo        13321  13347   commons 6955, osv5m 1915, youtube 4451
osaka         7175   7200   commons 5322, osv5m 341, youtube 1512
taipei        7235   7245   commons 5658, osv5m 533, youtube 1044
total 44142 rows
dropped: 115 사람사진과 음식 사진 그 두개는 어디 나라인지정도까진 추측이 가능해도 도시를 알기는 어려움 그리고
↪  의미없는 자동차 이런거 다 없앰
```

점수. balanced 0.5076 → balanced_drop 0.5099 (+0.0023). 서울은 0.496 → 0.536으로 올랐고 나머지 도시는 거의 그대로이거나 조금 내려갔다. 115장은 학습 사진의 0.26%이고, 차이는 seed 흔들림보다 작을 가능성이 크다. 결과를 어떻게 보는지(내 말): "아무래도 그 많은 사진 중에 115장 밖에 안뺐는데 당연히 정확도가 거의 그대로겠지 정말 다량의 사진을 검수 수거 하지 않는한 크게 안바뀔거야".

에이전트에게 준 요청. 사진을 고른 것과 규칙은 내가 정했고, 에이전트는 그 번호를 그대로 `curate.py drop`으로 뺐다.

```
[1] 사진을 보고 고른 뒤 에이전트에게 보낸 요청 (그대로)

seoul_00: 1 ,3, 5, 6 ,7 , 8 , 10, 11, 14 ,15, 18, 21, 27, 30, 31, 35, 36, 37 , 39 ,41 ,42, 43, 44,
↪  ,45 , 48
busan_00:1, 4, 5, 6 , 7, 8, 10, 12, 14, 16, 19, 21, 23, 24, 25, 26, 30, 31, 32,33, 34, 36, 39, 40,
↪  44, 45 ,46 ,47, 48
tokyo_00: 1, 3, 4, 7,10, 11, 12, 13,17, 18, 22, 26, 27, 29, 30 ,33, 34, 35 ,38, 40, 41, 43, 44, 45,
↪  46, 48
osaka_00: 2, 4, 7, 8,9, 11, 15, 16, 17, 18, 21, 22, 23, 24, 25, 29, 31, 32, 33, 34, 38, 39, 40, 42,
↪  44
taipei_00:11,12, 15,18,22,27,29,32,36,42

삭제해야할것

[2] 규칙(이유)을 묻자 보낸 답 (그대로)

사람사진과 음식 사진 그 두개는 어디 나라인지정도까진 추측이 가능해도 도시를 알기는 어려움 그리고 의미없는 자동차
↪  이런거 다 없앰

[3] 에이전트가 실행한 명령 (04_drop_suspects.sh 중, 도시마다 하나씩)

uv run python src/curate.py drop suspects_baseline_seed0_seoul_00 1 3 5 6 7 8 10 11 14 15 18 21 27
↪  30 31 35 36 37 39 41 42 43 44 45 48 --reason "<위 [2]의 문장>"
(busan, tokyo, osaka, taipei도 같은 방식)
uv run python src/curate.py count
```

Track B

tag	바뀐 것	validation
b1_mnv3l_320	첫 모델: MobileNetV3-Large(`IMAGENET1K_V2`), 320px, 6 epoch, 도시 균등(`--balance`), starter 사진만	0.5631
b2_mnv3l_384	이미지 크기만 320 → 384	0.5699

b1. `train_b.py` 기본값(ResNet18, 224px, 4 epoch)과 비교하면 모델, 이미지 크기, epoch, 균등 뽑기를 한꺼번에 바꿨기 때문에, 이 중 무엇이 도움이 됐는지는 b1만으로는 알 수 없다. 4.20M 파라미터, 0.44 GMACs. Track A의 balanced(0.508)보다 높게 나온 이유를 어떻게 보는지(내 말): "사진을 크게 보면 픽셀이 조각되면서 컴퓨터가 학습을 조금더 세밀하게할 수 있어서 더 높은 값을 추출했다".

b2. 위의 생각을 확인하려고 이미지 크기 하나만 384로 키웠다(0.63 GMACs). 0.5631 → 0.5699(+0.0068)로 올랐다. 서울(+0.032), 도쿄(+0.023)는 올랐고 부산(−0.017), 오사카(−0.017)는 내려갔다. 결과를 어떻게 보는지(내 말): "픽셀을 더 쪼개야한다고 봐". seed마다 다시 학습해 보지 않았으므로 +0.007이 크기 때문인지 우연인지는 확실하지 않다.

추가 사진. 모으지 않았다. 두 모델 모두 starter 학습 사진만 썼다.

검증 점수와 리더보드 점수. b1은 11월 9일 `track-b-1`, b2는 같은 날 23:37에 `track-b-2`로 릴리즈했다. 리더보드 점수는 보고서 작성 시점에 아직 공개되지 않았다. 두 점수가 다를 수 있는 이유: 검증 사진은 Wikimedia Commons, OSV-5M, YouTube가 섞인 starter와 같은 출처의 사진이지만, 리더보드 사진은 starter에 없는 채널의 YouTube 거리 영상 프레임이고, 최종 사진은 휴대폰으로 찍은 거리 사진이다. 학습 사진의 절반 이상이 Wikimedia Commons 사진이라, 거리 영상 프레임에서는 점수가 더 낮게 나올 수 있다.

채점기와 같은 방식으로 본 제출물 (b2)

```
/Users/rokaf_david/Documents/mas1004-assignment2/submission/model.onnx: 16.8 MB, 4.20 million
↪  parameters, 0.63 billion multiply-adds per 384 x 384 photo
  limits: 12 million parameters, 2.0 billion multiply-adds
  format and limits: OK

/Users/rokaf_david/Documents/mas1004-assignment2/data/starter/val: score 0.569 on 12077 photos
↪  (115s)
  seoul 0.584, busan 0.513, tokyo 0.630, osaka 0.470, taipei 0.650
  confusion (rows: true city, columns: answer):
    seoul    1422   256   336   163   258
    busan     399   713   117    64    97
    tokyo     424    85  2446   638   288
    osaka     176    93   690  1000   169
    taipei    312    66   233   175  1457
```

무엇이 차이를 만들었나

- Track A에서 가장 큰 변화는 balanced(+0.0115)였다. 사진을 빼기(+0.0023)는 거의 변화가 없었다. no_flip은 −0.0098로 오히려 내려갔다.
- Track B에서 이미지 크기 320 → 384는 +0.0068이었다.
- 두 트랙 사이의 차이(Track A 최고 0.510, Track B 최고 0.570)가 가장 컸지만, Track B는 모델, 이미지 크기, epoch을 함께 바꿨으므로 어느 것 때문인지는 따로 재지 않았다.
- 각 변경은 seed 0으로 한 번씩만 학습했다. baseline의 두 seed는 평균이 0.0005밖에 차이 나지 않았지만 도시별로는 0.024까지 움직였고, 흔들림을 두 번만 쟀으므로 balanced(+0.0115)처럼 큰 차이와 달리 115장 제외(+0.0023) 같은 작은 차이는 우연과 확실히 구분할 수 없다.

일주일이 더 있다면(내 말). "픽셀을 훨씬 더 많이 줄여서 이걸 반복해서 실험해보고 싶음 내 가설 증명하고 싶음".

All runs

```
finished,track,tag,seed,official_budget,budget,val_score,seoul,busan,tokyo,osaka,taipei,train_seco⌋
↪  nds,device,selection,recipe_sha,selection_sha,commit
2026-11-09T14:51:16,A,baseline,0,True,249984,0.4961,0.5951,0.3863,0.6241,0.3435,0.5314,3370.0,mps,⌋
↪  all training
↪  photos,17ac485223fa,,3ee1114
2026-11-09T16:08:13,A,balanced,0,True,249984,0.5076,0.4957,0.4712,0.5205,0.4347,0.6161,3722.1,mps,⌋
↪  all training
↪  photos,9a6cc745ea82,,d106946
2026-11-09T16:23:39,B,b1_mnv3l_320,,,265344,0.5631,0.554,0.5295,0.6083,0.4864,0.6371,3566.8,cuda,m⌋
↪  obilenet_v3_large 320px;
↪  data/starter/train,,,75f2eea
2026-11-09T20:58:21,A,balanced_drop,0,True,249984,0.5099,0.5359,0.459,0.5174,0.4234,0.6139,3491.7,⌋
↪  mps,"selection.csv, 44142
↪  rows",9a6cc745ea82,92442aae7b8b,73c703b
2026-11-09T21:15:45,B,b2_mnv3l_384,,,265344,0.5699,0.5864,0.5129,0.631,0.4699,0.6491,4361.3,cuda,m⌋
↪  obilenet_v3_large 384px;
↪  data/starter/train,,,4092bc6
```

```
2026-11-10T00:47:45,A,no_flip,0,True,249984,0.5001,0.5499,0.4302,0.5107,0.4192,0.5903,4124.9,mps,"selection.csv, 44142
rows",49dcfaa5e3ee,92442aae7b8b,8572dfd
2026-11-10T01:51:45,A,baseline,1,True,249984,0.4956,0.5979,0.3827,0.6068,0.3675,0.523,3796.3,mps,all training
photos,17ac485223fa,,32a1d97
```

Lab notebook

```
# Log

One entry for each recipe, selection or Track B model you train, newest at
the bottom. The numbers come from `experiments.csv`. "Why" and "what it means"
are in your own words: your agent asks you for them and does not make them up.
End the log with your plan for what to try next.

<!-- The form of an entry:

## <date> <tag> (Track A or B)
- changed: what is different from the run before
- why: your reason
- validation: 0.000 (before: 0.000)
- what it means: what you think the result says
-->

## 2026-11-09 Step 1: looking at the photos
- what tells the cities apart (my words): 서울 잠실 주경기장을 통해 알수 있음, 부산 - 해운대 모래사장을 통해 알 수 있음, 도쿄 - JR전철을 통해(초록색), 오사카 - 도톤보리 수로를 통해, 타이페이 - 한자와 타이페이101을 통해 알 수 있음
- sheets: results/curate/look_<city>_00.png (`curate.py look <city> --every 90`)

## 2026-11-09 baseline (Track A)
- changed: nothing; the starter recipe on all 44,257 training photos, seed 0
- why: the reference for every later Track A change
- validation: 0.496 (seoul 0.595, busan 0.386, tokyo 0.624, osaka 0.344, taipei 0.531)
- note: trained on this MacBook's GPU (mps) through scripts/official_mps.py, which runs src/official.py unchanged except that it turns off the channels_last memory format (torch 2.6 on mps stops with "view size is not compatible" otherwise)

## 2026-11-09 balanced (Track A)
- changed: sample_weights in track_a/recipe.py gives each photo 1 / (number of photos of its city), so every city is drawn equally often; nothing else changed
- why (my words): 어차피 최대 양의 사진 (도쿄)꺼를 해야하는데 그거만 13,347할게 아니라 그 시간에 부산도 여러번 보면 정확도가 올라갈거라 생각해서 밸런스 선택
- validation: 0.508 (before: 0.496); seoul 0.496, busan 0.471, tokyo 0.520, osaka 0.435, taipei 0.616
- what it means (my words): 서울 도쿄를 제외한 나머지 도시는 베이스라인보다 높은 정확도를 보여줬는데 그 이유는 아무래도 사진 양이 적었기에 여러번 학습하여 정확도가 올라갔다고 보여짐
- released as track-a-2

## 2026-11-09 b1_mnv3l_320 (Track B)
- changed: first Track B model: MobileNetV3-Large (torchvision IMAGENET1K_V2) at 320 x 320, 6 epochs, every city drawn equally often (--balance), starter training photos only; trained on Colab (cuda). 4.20 million parameters, 0.44 billion multiply-adds
- why: chosen from the options the agent explained: a smaller network that can look at a bigger image than ResNet18 at 224
- validation: 0.563 (seoul 0.554, busan 0.529, tokyo 0.608, osaka 0.486, taipei 0.637); check_submission.py on this Mac: format and limits OK, 0.562
- what it means (my words): 사진을 크게 보면 픽셀이 조각되면서 컴퓨터가 학습을 조금더 세밀하게할 수 있어서 더 높은 값을 추출했다
- note: this run changed the model, the image size, the epochs and the balancing at once compared with train_b.py as given, so it does not say which of them helped
- released as track-b-1
```

```
## 2026-11-09 balanced_drop (Track A), my decision from the photos
- looked at: the suspects sheets of the baseline (`curate.py suspects
↪  results/baseline_seed0/model.onnx --every 4`),
↪  results/curate/suspects_baseline_seed0_<city>_00.png, the 48 training photos per city the
↪  baseline thought least likely to be their city
- rule (my words, written before dropping): 사람사진과 음식 사진 그 두개는 어디 나라인지정도까진 추측이
↪  가능해도 도시를 알기는 어려움 그리고 의미없는 자동차 이런거 다 없앰
- changed: dropped the 115 photos I named (seoul 25, busan 29, tokyo 26, osaka 25, taipei 10) with
↪  `curate.py drop`; the selection went from 44,257 to 44,142 photos (track_a/dropped.csv); recipe
↪  unchanged (balanced)
- validation: 0.510 (before: 0.508); seoul 0.536, busan 0.459, tokyo 0.517, osaka 0.423, taipei
↪  0.614
- what it means (my words): 아무래도 그 많은 사진 중에 115장 밖에 안뺐는데 당연히 정확도가 거의 그대로겠지
↪  정말 다량의 사진을 검수 수거 하지 않는한 크게 안바뀔거야
- released as track-a-3

## 2026-11-09 b2_mnv3l_384 (Track B)
- changed: the image size only, 320 -> 384 (MobileNetV3-Large, 6 epochs, balanced, starter photos,
↪  as in b1); trained on Colab (cuda). 4.20 million parameters, 0.63 billion multiply-adds
- why (my words, from b1): 사진을 크게 보면 픽셀이 조각되면서 컴퓨터가 학습을 조금더 세밀하게할 수 있어서 더
↪  높은 값을 추출했다
- validation: 0.570 (before: 0.563); seoul 0.586, busan 0.513, tokyo 0.631, osaka 0.470, taipei
↪  0.649; check_submission.py on this Mac: format and limits OK, 0.569
- what it means (my words): 픽셀을 더 쪼개야한다고 봐
- released as track-b-2

## 2026-11-10 no_flip (Track A)
- changed: deleted T.RandomHorizontalFlip() from train_transform; selection (115 dropped) and
↪  everything else as in balanced_drop
- why (my words): 좌우반전을 하면 너 말대로 일본과 같은 반대인 나라에 대한 오류가 생길거라 생각이 들어
- why not the other options (my words): 나머지 옵션은 별로야 확대해봤자 픽셀이 더 세분화되는거 아니니 도움
↪  안될거같고 100%라는건 세상에 없어
- validation: 0.500 (before: 0.510); seoul 0.550, busan 0.430, tokyo 0.511, osaka 0.419, taipei
↪  0.590
- what it means (my words): 예상과 달리 떨어졌고, 이유는 모르겠다
- not released; track-a-3 stays my Track A submission, and track_a/recipe.py goes back to its
↪  version

## 2026-11-10 baseline, seed 1 (Track A)
- changed: only the seed, 0 -> 1; the starter recipe (from commit 2414a84, run with
↪  MAS1004_RECIPE_DIR=results/starter_recipe) on all 44,257 training photos
- why: to see how much the score moves by chance when nothing else changes
- validation: 0.4956 (seed 0: 0.4961); seoul 0.598, busan 0.383, tokyo 0.607, osaka 0.367, taipei
↪  0.523
- what it means (my words): 같은 설정인데도 값이 살짝 다른걸 보면 기계라고 그 전껄 100%같게 판독하지 못한다는
↪  걸 알았다. 사람처럼 얘네도 시행마다 살짝씩 다르단걸
- note: the commit column of this row is the commit at the time it ran (no_flip); the recipe_sha is
↪  the starter recipe's (17ac485223fa, the same as seed 0)

## What I would try next (my words)
- 픽셀을 훨씬 더 많이 줄여서 이걸 반복해서 실험해보고 싶음 내 가설 증명하고 싶음
```